SSMM — Software Scope Management Module

SSMM — Software Scope Management Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Requirements Standard (SRS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Scope Management Governance

Module ID: SLGA-SRS-SSMM-003

Origin ID: OOF.CM.SLGA.01.03.01

Version: 1.0

Status: Canonical Draft · Open Standard

Effective Date: Pending Canonical Publication

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Software Scope Management Module (SSMM) governs how the scope of

software requirements is identified, defined, bounded, evaluated, accepted, and maintained across software objects, capabilities, configurations, operating contexts, and lifecycle conditions.

SSMM establishes the methodological conditions under which a specified

software requirement may be recognized as applicable to a particular software scope.

It distinguishes the existence of a requirement from its applicability, the

recognition of a software purpose from authorization of its implementation scope, and the technical reach of a software capability from the boundaries within which that capability is legitimately intended to operate.

SSMM applies regardless of whether software scope is defined by humans, organizations, AI agents, autonomous systems, automated development environments, or combinations of these actors.

It preserves scope boundaries when requirements are distributed across components, teams, environments, versions, and software development processes.

Its principal output is:

Governed Software Scope State

Its foundational distinction is:

Requirement Acceptance ≠ Requirement Scope Applicability.

Operational Role

The operational role of SSMM is:

To establish and maintain an identifiable, justified, and traceable

software requirements scope that determines where accepted or proposed requirements apply, where they do not apply, and which conditions must be satisfied before scope expansion, restriction, or reassignment can be recognized.

SSMM prevents a requirement from being treated as universally applicable

merely because it has been specified, accepted, or implemented somewhere within a software system.

It provides the applicability foundation required for governed requirements

baseline formation.

SSMM does not independently authorize software creation, requirements changes, release, or deployment.

Position in the SRS Internal Governance Flow

The SRS internal governance flow is:

Software Purpose Definition → Requirements Specification → Software Scope

Management → Requirements Change Control → Requirements Traceability

SSMM governs:

Software Scope Management

It is the third Core Module of SRS.

Its principal upstream inputs are:

Governed Software Purpose State

from:

SPDM — Software Purpose Definition Module

and:

Governed Software Requirements Specification State

from:

RSPM — Requirements Specification Module

Its principal output is:

Governed Software Scope State

That state provides the applicable scope reference for:

RCCM — Requirements Change Control Module

and supports subsequent:

RTM — Requirements Traceability Module

Central Operational Question

Which software requirements legitimately apply to which software objects,

functions, configurations, environments, actors, and lifecycle conditions, and how can those applicability boundaries remain explicit when software evolves or its development becomes distributed and autonomous?

Software Scope Governance Object

Every SSMM assessment must identify the Software Scope Governance Object.

The object represents the software capability, system, component, service,

or defined software context whose requirements applicability is being governed.

It should identify, where applicable:

• Software object identity

• Relevant purpose reference

• Applicable requirements set

• Scope version or reference state

• Software boundaries

• Relevant configurations

• Intended operating contexts

• Scope origin and recognition conditions

• Material dependencies and exclusions

A software system may contain multiple overlapping scope objects.

Scope shall therefore not be inferred solely from a project name,

repository, or application boundary.

Software Object Identity ≠ Software Scope.

Scope Origin and Recognition Basis

SSMM shall identify how a proposed software scope originated.

Possible sources include:

• Organizational project mandates

• Product or service definitions

• Governed software purpose states

• Accepted requirements

• Contractual or operational commitments

• Applicable external constraints

• Software modernization initiatives

• AI-generated scope proposals

• Autonomous development plans

The origin of a scope proposal shall remain traceable.

However, a proposed scope does not become legitimate merely because it originates from an authorized project or competent technical system.

Scope Origin ≠ Scope Authorization.

Where recognition authority is unclear, the resulting scope shall remain conditional or unresolved.

Scope Definition and Intended Coverage

SSMM shall establish what the governed software scope includes.

Relevant dimensions may include:

Functional Coverage: The capabilities and behaviors included within the scope.

Software Object Coverage: The systems, services, components, and

interfaces concerned.

Requirements Coverage: The requirements intended to apply.

Operational Coverage: The relevant operating contexts and conditions.

Configuration Coverage: The configurations to which requirements apply.

Lifecycle Coverage: The software lifecycle stages or states relevant to

the scope.

Organizational Coverage: The applicable organizational or contractual

boundaries.

Not every software scope requires every dimension.

The necessary dimensions depend on the governed software reality.

A scope definition shall be sufficiently explicit to distinguish inclusion

from assumption.

Scope Inclusion and Exclusion

SSMM shall identify material inclusions and exclusions.

An inclusion establishes that a defined requirement, function, or software object belongs within the proposed or recognized scope under specified conditions.

An exclusion establishes that a defined element is outside that scope.

Exclusions should be explicit where omission could create material ambiguity.

A scope may exclude:

• Particular software components

• Specific user groups

• Certain operating environments

• Optional capabilities

• Legacy functions

• External systems

• Particular configurations

• Unapproved autonomous functions

An exclusion shall not be used to conceal a requirement that remains mandatory under a competent external obligation.

Scope Exclusion ≠ Elimination of Applicable Obligation.

Where an exclusion conflicts with an applicable obligation, the conflict shall be preserved for competent resolution.

Requirement Applicability

SSMM shall evaluate whether each material requirement applies to the

identified Software Scope Governance Object.

Applicability may depend on:

• Software purpose

• Requirement acceptance

• Component identity

• Intended function

• Configuration

• Operating environment

• Applicable users or actors

• Relevant external obligations

• Version or lifecycle state

• Explicit applicability conditions

A requirement may be valid and accepted without applying to every software object.

Conversely, a requirement may be materially applicable to a software object even where the current project documentation has omitted it.

Requirement Existence ≠ Requirement Applicability.

Requirement Omission ≠ Non-Applicability.

Scope–Purpose Alignment

SSMM shall evaluate the relationship between the proposed software scope

and the applicable Governed Software Purpose State.

The assessment should determine whether the scope:

• Supports the recognized software purpose

• Introduces capabilities beyond that purpose

• Excludes capabilities necessary for that purpose

• Changes intended use

• Creates material purpose conflicts

• Depends on unresolved purpose conditions

Where the proposed scope materially changes the recognized software purpose, SSMM shall refer the matter to SPDM.

SSMM does not independently redefine the software purpose.

Purpose Recognition ≠ Unlimited Scope Authorization.

Scope Completeness and Sufficiency

SSMM shall assess whether the defined scope adequately represents the

material requirements applicable to the software object and intended context.

Scope completeness concerns whether material applicability dimensions have

been addressed.

Scope sufficiency concerns whether the established boundaries are adequate for the intended governance use.

A scope may be clearly bounded while excluding a capability essential to the recognized purpose.

A scope may also be broad enough to include relevant functions but insufficiently precise to establish which requirements apply to each component.

Defined Scope ≠ Sufficient Scope.

Requirements Completeness ≠ Scope Completeness.

RSPM governs requirements completeness against the software purpose.

SSMM governs the applicability and coverage of those requirements within defined software boundaries.

Scope Granularity and Hierarchy

Software scope may exist at multiple levels.

Examples include:

• Software portfolio

• Product

• System

• Service

• Subsystem

• Component

• Interface

• Function

• Configuration

• Specific operating context

SSMM shall identify material parent–child and overlapping scope

relationships.

A requirement applicable to a parent scope shall not automatically be

assumed to apply identically to every subordinate scope.

Likewise, requirements established for one component shall not

automatically govern an entire system.

Parent Scope ≠ Automatic Universal Applicability.

Scope inheritance must preserve the conditions under which a requirement applies.

Scope Dependencies and Shared Boundaries

Software scopes may depend on shared components, interfaces, services, data, or operational capabilities.

SSMM shall identify dependencies that materially affect requirements applicability.

Examples include:

• Shared authentication services

• Common data interfaces

• Third-party software services

• Shared infrastructure functions

• Common operational controls

• Dependencies between software components

A dependency may create requirements implications beyond the immediately modified component.

However, the existence of a technical dependency does not automatically authorize expansion of the governed scope.

Technical Dependency ≠ Scope Authorization.

Detailed software dependency governance remains within the competent SLGA™ Software Dependency Standard.

Scope Configuration and Environment Alignment

A requirement may apply differently across software configurations and environments.

SSMM shall preserve material distinctions between:

• Development environments

• Testing environments

• Staging environments

• Production environments

• Regional or organizational deployments

• Configuration variants

• Product editions

• Legacy and current versions

A requirement accepted for one configuration shall not automatically be

treated as applicable to another.

Configuration Similarity ≠ Scope Equivalence.

Test Scope ≠ Production Scope by default.

Where requirements applicability depends on configuration, the relevant conditions shall remain identifiable.

Scope Boundary Across Software Versions

Software versions may introduce, modify, or retire capabilities.

SSMM shall determine whether an existing scope remains applicable when the software version changes.

Relevant considerations include:

• Changes in functionality

• New or removed components

• Modified interfaces

• Changed operating assumptions

• Updated requirements

• New applicable constraints

• Changes in intended use

A version change does not necessarily require a new scope.

However, material changes affecting applicability shall trigger scope

reassessment.

Version Continuity ≠ Scope Continuity by default.

Scope Under Distributed Development

Software may be developed by multiple teams, suppliers, organizations, and autonomous systems.

SSMM shall preserve scope identity and applicability across distributed

development activities.

The governance process should identify:

• Which software objects belong to each governed scope

• Which requirements apply to each object

• Where scope boundaries overlap

• Which interfaces connect different scopes

• Where interpretations conflict

• Which changes require coordination

Distributed development shall not cause locally accepted requirements to be

treated as globally applicable without sufficient governance justification.

Local Scope Recognition ≠ Global Scope Applicability.

Autonomous Scope Expansion

AI agents and autonomous development systems may identify opportunities to extend software functionality.

Such systems may propose:

• New features

• Additional data access

• New integrations

• Expanded automation

• Additional tool permissions

• Changes to operating boundaries

• New downstream actions

SSMM shall distinguish proposed scope expansion from legitimately

recognized scope expansion.

An agent's authority to complete a development objective does not automatically authorize every additional function or tool path it proposes.

Task Authority ≠ Constituent Action Authority.

Autonomous Capability Discovery ≠ Authorized Scope Expansion.

Material expansion proposals shall remain subject to the applicable

purpose, requirements, authority, and change-governance conditions.

Scope Conflict and Boundary Overlap

Different requirements or software scopes may overlap or conflict.

Examples include:

• Two components claiming responsibility for the same function

• Different teams assigning incompatible requirements to a shared

• A feature included in one scope but excluded from another

• A software requirement applying under one configuration but not

• Conflicting organizational or contractual scope interpretations

interface. another.

SSMM shall identify material conflicts and distinguish:

Compatible Overlap

Conditional Overlap

Conflicting Overlap

Unresolved Boundary

Where conflict resolution requires competent authority, SSMM shall preserve the unresolved condition rather than impose an unsupported resolution.

Scope Overlap ≠ Scope Agreement.

Scope Evidence and Recognition Conditions

SSMM shall preserve the evidence supporting material scope determinations.

Relevant evidence may include:

• Governed software purpose references

• Accepted requirement references

• Scope proposals

• Competent recognition records

• Contractual or organizational boundaries

• Applicable external constraints

• Configuration records

• Interface definitions

• Scope assessment findings

• Exclusion justifications

Scope recognition shall preserve its applicable conditions.

A scope should not be represented as unconditionally recognized where its authority, evidence, or applicability remains materially unresolved.

Documented Scope ≠ Governed Scope.

Technical Scope Description ≠ Evidence of Scope Authorization.

Scope Recognition and Governance States

SSMM shall represent the current governance condition of a software scope.

Possible states include:

Proposed: The scope has been submitted or generated but not sufficiently

evaluated.

Under Evaluation: The scope is undergoing applicability, boundary, or recognition assessment.

Recognized: The scope satisfies applicable recognition conditions.

Conditionally Recognized: The scope is recognized subject to explicit limitations or unresolved conditions.

Partially Defined: Material scope dimensions remain insufficiently

specified.

Conflicted: Material scope contradictions remain unresolved.

Evidence-Limited: Available evidence is insufficient for a reliable scope determination.

Out of Scope: A requirement or capability has been determined not to

apply within the defined context.

Unresolved: The available information does not support a sufficiently determined state.

Superseded: A later governed scope state has replaced the previous state.

Withdrawn: The scope has been withdrawn through an applicable competent process.

These states describe the governance condition of the scope.

They do not independently authorize implementation or deployment.

Recognized Scope ≠ Software Execution Authorization.

Requirements Baseline Applicability

SSMM shall support determination of which accepted requirements are applicable to a defined software baseline.

The process shall distinguish:

Specified Requirement

from:

Accepted Requirement

from:

Applicable Baseline Requirement

A requirement may be accepted generally but not applicable to a particular software object, configuration, or version.

A requirement may also be applicable only when a defined condition is satisfied.

SSMM shall preserve those applicability conditions.

The resulting scope state provides an essential input to the Governed

Software Requirements Baseline established under SRS.

Accepted Requirement ≠ Applicable Baseline Requirement.

Requirements Baseline Inclusion Requires Governed Applicability.

Scope Change and Impact

Software scope may change because of:

• New requirements

• Removed requirements

• Changes in software purpose

• New integrations

• Component restructuring

• Configuration changes

• Organizational changes

• Changes in applicable obligations

• AI-generated expansion proposals

• Operational findings

SSMM shall identify whether a proposed modification affects the recognized

scope.

Material scope changes should preserve:

• Previous scope identity

• Proposed new scope

• Change origin

• Change justification

• Affected requirements

• Affected software objects

• Material dependencies

• Applicable authority conditions

• Recognition state

• Relevant impact findings

RCCM governs the competent requirements change-control process.

SSMM governs the resulting scope and applicability implications.

Scope Change Proposal ≠ Authorized Scope Change.

Scope Drift and Unauthorized Expansion

Scope Drift occurs when the actual or proposed software scope progressively diverges from the recognized scope without adequate governance recognition.

Potential indicators include:

• Features added without scope reassessment

• Requirements applied to unintended components

• Autonomous agents introducing additional functions

• Unreviewed expansion of data access

• New integrations outside the recognized boundary

• Informal assumptions replacing explicit applicability conditions

SSMM shall identify material scope drift and determine whether reassessment

is required.

Technical implementation of an expanded capability does not retroactively legitimize the expansion.

Implemented Functionality ≠ Recognized Scope.

Scope Continuity and Reassessment

A governed software scope shall remain identifiable when development actors, software versions, configurations, organizations, or operating environments change.

SSMM shall preserve sufficient continuity to reconstruct:

• Previous scope states

• Material boundary changes

• Applicable requirements

• Recognition conditions

• Affected software objects

• Unresolved conflicts

• Reassessment outcomes

Reassessment may be triggered by material changes in purpose, requirements, configuration, dependencies, operational context, or applicable obligations.

A previously recognized scope shall not automatically remain valid under

materially changed conditions.

Historical Scope Recognition ≠ Permanent Scope Validity.

Governed Software Scope State

The principal output of SSMM is: Governed Software Scope State

The state shall preserve sufficient information to identify:

Software Scope Governance Object: The software object or context whose

scope is governed.

Scope Identity: The identifiable scope and relevant version.

Purpose Reference: The applicable Governed Software Purpose State.

Requirements Reference: The relevant Governed Software Requirements

Specification State.

Scope Definition: The included functions, software objects, and operating

contexts.

Scope Exclusions: Material functions, objects, or contexts excluded from

the scope.

Requirements Applicability: Requirements applicable to the defined scope.

Conditional Applicability: Requirements that apply only under specified conditions.

Scope Hierarchy: Material parent–child and overlapping relationships.

Configuration and Environment: Relevant software variants and operating

contexts.

Scope Dependencies: Material interfaces and shared boundaries.

Scope Recognition: The applicable governance state and recognition

conditions.

Scope Conflicts: Material unresolved overlaps or contradictions.

Evidence: Supporting sources and material limitations.

Uncertainty: Unresolved applicability or boundary conditions.

Change and Continuity: Relevant scope history and reassessment conditions.

Traceability: Relationships to software purpose, requirements, and

subsequent change governance.

The state shall be sufficiently explicit to support requirements baseline

applicability and subsequent requirements change governance.

Minimum Implementation Framework

A conforming implementation of SSMM must provide the following six functional capabilities.

• Software Scope Object and Boundary Governance

Capability to identify the Software Scope Governance Object, define scope

identity, and establish material inclusions, exclusions, contexts, and boundaries.

The implementation must distinguish software identity from governed software scope.

• Requirements Applicability and Purpose Alignment Governance

Capability to evaluate which requirements apply to the defined scope and

whether the scope remains aligned with the applicable Governed Software Purpose State.

The implementation must distinguish accepted requirements from applicable baseline requirements.

• Scope Hierarchy, Configuration, and Dependency Governance

Capability to represent scope relationships across software systems,

components, interfaces, versions, configurations, environments, and material dependencies.

The implementation must prevent unsupported inheritance of applicability across different scopes.

• Scope Conflict, Evidence, and Recognition Governance

Capability to identify overlapping or conflicting scopes, preserve

supporting evidence, assess recognition conditions, and represent unresolved applicability or authority questions.

The implementation must not convert technical inclusion into legitimate scope authorization.

• Scope Change, Drift, and Continuity Governance

Capability to identify material scope changes, detect unauthorized

expansion, preserve scope history, evaluate affected requirements, and trigger reassessment.

The implementation must distinguish proposed changes from changes authorized through the applicable competent process.

• Governed Scope State and Requirements Change Handoff Governance

Capability to produce a Governed Software Scope State containing sufficient

applicability, boundary, recognition, evidence, and continuity information for subsequent requirements change governance and baseline maintenance.

The implementation must preserve material unresolved conditions during handoff.

GIE™ Compatibility

SSMM is compatible with GIE™ as a downstream methodological execution and analysis layer.

GIE™ may support:

• Software scope discovery

• Requirements applicability mapping

• Purpose–scope alignment assessment

• Scope hierarchy representation

• Boundary overlap and conflict detection

• Configuration and environment comparison

• Scope drift identification

• Change-impact analysis

• Evidence and recognition-state mapping

• Governed scope state generation

• Reassessment and downstream handoff

GIE™ shall not independently grant scope authority, expand the recognized

scope, or convert technical reach into legitimate applicability.

Automated scope findings must remain distinguishable from competent governance determinations.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical SSMM governance logic is:

Receive Governed Software Purpose State

↓

Receive Governed Software Requirements Specification State

↓

Identify Software Scope Governance Object

↓

Establish Scope Identity and Context

↓

Identify Scope Origin and Recognition Basis

↓

Define Scope Inclusions and Exclusions

↓

Identify Relevant Software Objects and Functions

↓

Map Requirements to Software Scope

↓

Assess Purpose–Scope Alignment

↓

Assess Requirements Applicability

↓

Identify Scope Hierarchy and Dependencies

↓

Assess Configuration and Environment Conditions

↓

Assess Scope Completeness and Sufficiency

↓

Identify Boundary Overlaps and Conflicts

↓

Assess Evidence and Recognition Conditions

↓

Preserve Unresolved Applicability and Authority Questions

↓

Determine Scope Governance State

↓

Establish Governed Software Scope State

↓

Preserve Scope Identity, Versions, and Traceability

↓

Handoff to Requirements Change Control

↓

Reassess When Material Scope Conditions Change

The principal output is:

Governed Software Scope State

Governance Outputs

SSMM may produce:

- Software Scope Governance Object

- Software Scope Identity Record - Scope Origin Record - Scope Definition - Scope Inclusion Record - Scope Exclusion Record - Purpose–Scope Alignment State - Requirements Applicability State - Conditional Applicability Record - Scope Completeness Assessment - Scope Sufficiency Assessment - Scope Hierarchy Record - Scope Dependency Record - Configuration Applicability Record - Environment Applicability Record - Scope Boundary Overlap Record - Scope Conflict Record - Scope Recognition State - Scope Recognition Conditions - Scope Evidence State - Scope Uncertainty Record - Scope Change Record - Scope Impact Assessment - Scope Drift Record - Scope Continuity Record - Scope Reassessment Record - Requirements Baseline Applicability Record - Requirements Change Handoff Record

The principal output is:

Governed Software Scope State

Critical Distinctions

SSMM preserves the following distinctions:

Requirement Acceptance ≠ Requirement Scope Applicability

Software Object Identity ≠ Software Scope

Scope Origin ≠ Scope Authorization

Scope Exclusion ≠ Elimination of Applicable Obligation

Requirement Existence ≠ Requirement Applicability

Requirement Omission ≠ Non-Applicability

Purpose Recognition ≠ Unlimited Scope Authorization

Defined Scope ≠ Sufficient Scope

Requirements Completeness ≠ Scope Completeness

Parent Scope ≠ Automatic Universal Applicability

Technical Dependency ≠ Scope Authorization

Configuration Similarity ≠ Scope Equivalence

Test Scope ≠ Production Scope by default

Version Continuity ≠ Scope Continuity by default

Local Scope Recognition ≠ Global Scope Applicability

Task Authority ≠ Constituent Action Authority

Autonomous Capability Discovery ≠ Authorized Scope Expansion

Scope Overlap ≠ Scope Agreement

Documented Scope ≠ Governed Scope

Technical Scope Description ≠ Evidence of Scope Authorization

Recognized Scope ≠ Software Execution Authorization

Accepted Requirement ≠ Applicable Baseline Requirement

Scope Change Proposal ≠ Authorized Scope Change

Implemented Functionality ≠ Recognized Scope

Historical Scope Recognition ≠ Permanent Scope Validity

Cross-Architecture Boundary

SSMM governs software scope and requirements applicability within SRS and SLGA™.

It does not replace competent governance of authority, accountability, operational reality, software dependencies, configuration, software creation, or autonomous system execution.

Relevant interoperating architectures may include:

GOA™ — Governance Architecture: General authority, mandate, and

delegation boundaries.

AGA™ — Accountability Governance Architecture: Applicable responsibility

and accountability conditions.

ASGA™ — Autonomous Systems Governance Architecture: Autonomous system

authority and execution boundaries.

ORA™ — Operational Reality Architecture: Competent assessment of

operational reality conditions.

VALIDOS™: Applicable validation methodology.

ADIT®: Broader evidence and auditability governance.

Within SRS:

SPDM governs software purpose definition.

RSPM governs requirements specification and completeness assessment.

SSMM governs software scope and requirements applicability.

RCCM governs requirements changes and their competent control.

RTM governs requirements traceability.

Within SLGA™:

The Software Dependency Standard governs the software dependency

lifecycle.

The Software Configuration Standard governs software configuration states.

The Software Creation Standard governs software creation activities.

The Software Lifecycle Control Standard governs cross-cutting lifecycle

control.

SSMM may consume relevant governed states from these domains without replacing their responsibilities.

Canonical Boundary

SSMM begins when software requirements must be assigned to a defined software object, capability, configuration, operating context, or lifecycle scope.

SSMM governs:

• Software scope object identification

• Scope identity

• Scope origin and recognition basis

• Scope definition

• Scope inclusion and exclusion

• Requirements applicability

• Purpose–scope alignment

• Scope completeness and sufficiency

• Scope hierarchy

• Scope dependencies

• Configuration and environment applicability

• Scope version alignment

• Distributed development boundaries

• Autonomous scope expansion

• Scope overlap and conflict

• Scope evidence

• Scope recognition

• Requirements baseline applicability

• Scope change implications

• Scope drift

• Scope continuity

• Scope reassessment

• Requirements change handoff

SSMM does not independently determine:

• General organizational or legal authority

• Recognition of a materially new software purpose

• Requirements specification completeness

• Authorization of substantive requirements changes

• Software design or implementation

• Software dependency lifecycle validity

• Software configuration validity

• Software quality or security conformance

• Release or deployment authorization

• Complete lifecycle traceability governance

SSMM ends when the relevant software requirements scope has been represented as a:

Governed Software Scope State

with sufficient identity, applicability, boundaries, recognition

conditions, evidence, and continuity to support subsequent requirements change governance and baseline maintenance.

• Use Case 1 — A Requirement Is Accepted, but It Does Not Apply Everywhere

A software company develops a platform used by several business units.

A requirement is accepted requiring all customer transactions to undergo an additional approval step.

The requirement is legitimate for one business unit handling high-value financial transactions.

However, another business unit uses the same platform for low-risk internal

requests.

The development team initially assumes that the accepted requirement applies across the entire application.

SSMM evaluates the requirement against the relevant software scope.

The assessment identifies:

Requirement Identity

Applicable Business Unit

Applicable Transaction Type

Relevant Software Components

Configuration Conditions

Scope Exclusions

The resulting Governed Software Scope State establishes that the requirement applies only under the recognized conditions.

The requirement remains accepted.

Its applicability is bounded.

Accepted Requirement ≠ Universal Requirement Applicability.

SSMM prevents a locally applicable requirement from becoming an unsupported

global software obligation.

Use Case 2 — An AI Agent Expands the Software Beyond Its Assigned Scope

A company uses an autonomous coding agent to improve an internal inventory

application.

The authorized development objective is to optimize stock-level reporting.

During implementation planning, the agent identifies an opportunity to

integrate supplier purchasing systems and automatically place replenishment orders.

The proposed integration may be technically useful.

However, it introduces:

• New external system connections

• Additional data access

• Financially consequential actions

• New operational dependencies

• Capabilities outside the original development scope

SSMM distinguishes the authorized reporting scope from the proposed

purchasing automation scope.

The additional capability remains a proposed scope expansion.

The module identifies the affected software objects, requirements,

dependencies, and applicable recognition conditions.

The resulting scope state may remain:

Conditionally Recognized

or:

Unresolved

until the competent governance processes determine whether the expansion is legitimate.

The agent's ability to implement the integration does not establish authority to expand the software's purpose or scope.

Autonomous Capability Discovery ≠ Authorized Scope Expansion.

Task Authority ≠ Constituent Action Authority.

Canonical Closing Statement

Software scope determines where requirements apply and where their applicability ends.

A requirement may be correctly specified and legitimately accepted without applying to every software component, configuration, environment, or operating context.

A software capability may be technically available without belonging to the recognized scope.

A project may have an approved purpose without possessing unlimited authority to expand its functionality.

And an autonomous development system may identify useful extensions without possessing authority to incorporate them.

Requirement Acceptance ≠ Requirement Scope Applicability.

Purpose Recognition ≠ Unlimited Scope Authorization.

Technical Dependency ≠ Scope Authorization.

Implemented Functionality ≠ Recognized Scope.

Autonomous Capability Discovery ≠ Authorized Scope Expansion.

SSMM establishes the methodological conditions under which software scope

becomes identifiable, bounded, justified, and traceable.

It preserves applicability across software objects, versions,

configurations, development teams, and autonomous actors.

It ensures that scope changes remain distinguishable from technical changes and that expanded functionality does not silently redefine the boundaries of the software requirements baseline.

SSMM does not determine whether the software should be implemented or deployed.

It establishes where the governed requirements legitimately apply.

SSMM transforms software scope from an assumed project boundary into a

governed requirements applicability state — ensuring that the reach of software functionality never substitutes for the legitimacy of the scope within which that functionality is intended to operate.